
Государственное бюджетное профессиональное образовательное учреждение
«Академия Инженерных Технологий и Управления»

МЕТОДИЧЕСКОЕ ПОСОБИЕ

по выполнению практического задания

Модуль 1.

Участие в проектировании архитектуры интеллектуальных интегрированных систем

Специальность: 09.02.08

Интеллектуальные интегрированные системы

Форма обучения: очная

Составлено:

Преподаватель ПМ.03

_____/_____
(подпись) (Ф.И.О.)

Утверждено:

Председатель ПЦК специальности

_____/_____
(подпись) (Ф.И.О.)

2026

Аннотация

Настоящее методическое пособие предназначено для студентов, выполняющих практическое задание по Модулю 1 «Участие в проектировании архитектуры интеллектуальных интегрированных систем» в рамках специальности 09.02.08 «Интеллектуальные Интегрированные Системы».

Пособие содержит теоретические сведения, необходимые для понимания принципов построения графических приложений на языке Python с применением библиотеки PySide6, а также описание архитектурных решений, использованных при разработке проекта «Интеллектуальная Дорожная Система».

Целевое назначение данного документа — сформировать у студента глубокое понимание задания модуля. Пособие не является пошаговой инструкцией с готовыми ответами: оно объясняет концептуальную основу каждого решения, что позволяет студенту самостоятельно реализовать задание и успешно представить решение.

Содержание

Аннотация	2
Содержание	3
1. Архитектура приложения	4
1.1. Основные классы приложения	4
1.2. Архитектурная модель	4
2. Написание кода приложения	5
2.1. Создание точки входа	5
2.2. Создание главного окна	5
2.2.1. Настройка окна	5
2.2.2. Компоновка интерфейса	6
2.2.3. Сохранение и загрузка	6
2.3. Константы и утилиты	7
2.3.1. Константы	7
2.3.2. Утилиты	8
2.4. Реализация сцены — класс World	10
2.4.1. Поиск и навигация по сцене	10
2.4.2. Обработка нажатия по сцене	10
2.4.3. Отрисовка сетки	11
2.4.4. Обновление сущностей по таймеру	11
2.4.5. Генерация случайного маршрута	12
2.5. Базовый класс сущности — BaseEntity	13
2.5.1. Инициализация	13
2.5.2. Поведение по таймеру	14
2.5.3. Обработка перемещения	14
2.5.4. Установка позиции	15
2.5.5. Валидация размещения	15
2.6. Сериализация и загрузка сцены – модуль persistence	15
2.6.1. Сериализаторы сущностей	16
2.6.2. Регистр сериализаторов	16
2.6.3. Сериализация сущности	17
2.6.4. Десериализация сущности	17
2.6.5. Сохранение сцены	18
2.6.6. Загрузка сцены	18

1. Архитектура приложения

Архитектура определяет распределение ответственности между компонентами системы: инкапсуляцию данных, зоны управления и граф взаимодействий.

1.1. Основные классы приложения

Класс	Наследуется от	Роль в приложении
MainWindow	MainWindow	Главное окно. Осуществляет компоновку виджетов, маршрутизацию сигналов между панелями управления и сценой (World)
World	QGraphicsScene	Сцена, хранит сущности, предоставляет методы для получения информации о мире
BaseEntity	QGraphicsPixmapItem	Базовый класс для всех объектов на сцене. Управляет позицией (клетка), реализует проверку корректности размещения и обработкой событий
ControlPanel	QDockWidget	Панель управления, предоставляет инструменты для взаимодействия с миром
PropertiesPanel	QDockWidget	Отображает свойства выбранной сущности и позволяет удалить её со сцены

1.2. Архитектурная модель

Приложение построено по схеме «Сцена — Сущности — Контроллер»:

- Сцена (World) — контейнер, хранящий все объекты и предоставляющий методы для навигации и поиска
- Сущности (BaseEntity и дочерние классы) — самодостаточные объекты, содержащие логику своего поведения
- Контроллер (MainWindow) — связывает UI, сцену и панели, обрабатывает пользовательские действия

Сущности не хранят ссылок на другие сущности. Вся информация о соседях получается через методы сцены. Это делает сущности переиспользуемыми и упрощает тестирование.

2. Написание кода приложения

2.1. Создание точки входа

Точка входа — минимальный скрипт, который создаёт приложение и главное окно (Листинг 1).

Точка входа — скрипт, который инициализирует Qt-приложение и создаёт главное окно. Передача `sys.argv` в `QApplication` позволяет обрабатывать аргументы командной строки, а вызов `exec()` запускает цикл обработки событий.

```
import sys
from PySide6.QtWidgets import QApplication
from src.window import MainWindow

if __name__ == "__main__":
    app = QApplication(sys.argv)
    main_window = MainWindow(app)
    main_window.show()
    app.exec()
```

Python

Листинг 1. Точка входа в приложение

2.2. Создание главного окна

`MainWindow` наследуется от `QMainWindow` и выступает контроллером приложения, связывая сцену, панели управления и пользовательские действия.

2.2.1. Настройка окна

Метод `setupWindow` (Листинг 2) задаёт заголовок, размеры окна, и размечает меню приложения.

```
def setupWindow(self) -> None:
    self.setWindowTitle("ИИС Дорожной системы")
    self.resize(1100, 700)

    menu = self.menuBar().addMenu("Файл")
    menu.addAction("Сохранить").triggered.connect(self.save)
    menu.addAction("Загрузить").triggered.connect(self.load)

    self.menuBar().addAction("События").triggered.connect(
        self.openEventsWindow
    )
```

Python

Листинг 2. Настройка окна

2.2.2. Компоновка интерфейса

`setLayout` (Листинг 3) создаёт сцену (`World`) и помещает её в центральный виджет (`WorldView`). Слева добавляется панель управления (`ControlPanel`), справа — панель свойств (`PropertiesPanel`).

Здесь также настраиваются сигналы: клик по клетке вызывает добавление сущности, выбор объекта обновляет панель свойств, а действия с панели управления управляют режимом светофоров и добавлением машин на сцену.

```
def setupLayout(self) -> None:
    self.world = World()
    self.world.onWorldCellClicked.connect(self.onWorldCellClicked)
    self.world.selectionChanged.connect(self.onSelectionChanged)
    self.setCentralWidget(WorldView(self.world))

    self.controlPanel = ControlPanel(self)
    self.controlPanel.entityFactorySelected.connect(
        self.onFactorySelected
    )

    self.controlPanel.spawnCarsClicked.connect(self.onSpawnCarsClicked)
    self.controlPanel.tlModeChanged.connect(self.onTlModeChanged)
    self.addDockWidget(
        Qt.DockWidgetArea.LeftDockWidgetArea,
        self.controlPanel
    )

    self.propertiesPanel = PropertiesPanel()
    self.addDockWidget(
        Qt.DockWidgetArea.RightDockWidgetArea,
        self.propertiesPanel
    )
```

Python

Листинг 3. Компоновка интерфейса

2.2.3. Сохранение и загрузка

Методы `save` и `load` (листинг 6) открывают диалог выбора файла и вызывают утилиты `saveWorld` / `loadWorld`, которые сериализуют состояние сцены в JSON и восстанавливают его оттуда.

```
def save(self):
    path, _ = QFileDialog.getSaveFileName(
        self, "Сохранить", "", "JSON (*.json)"
    )
    if path:
        saveWorld(self.world, path)

def load(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "", "Загрузить", "JSON (*.json)"
    )
    if path:
        loadWorld(self.world, path)
```

Python

Листинг 4. Методы сохранения и загрузки

2.3. Константы и утилиты

Перед реализацией сцены необходимо определить базовые константы и вспомогательные функции, которые используются всеми компонентами приложения.

2.3.1. Константы

`SRC_PATH`, `BASE_PATH` и `MEDIA_PATH` (Листинг 5) определяют структуру каталогов проекта с помощью объектно ориентированной, мультиплатформенной библиотеки работы с путями для работы на любой машине.

```
SRC_PATH = Path(__file__).parent
BASE_PATH = SRC_PATH.parent
MEDIA_PATH = BASE_PATH / "media"
```

Python

Листинг 5. Пути к файлам

`CELL_SIZE` задаёт размер клетки в пикселях (35×35), а `GRID_DIM` определяет размер сетки 50×50 клеток (Листинг 6)

```
CELL_SIZE = 35
GRID_DIM = 50
```

Python

Листинг 6. Параметры сетки

Словарь `DIR2ROT` сопоставляет направления с углами поворота текстур (в градусах). `DIR2OFFSET` определяет смещения по координатной сетке: движение на север уменьшает `Y`, на юг — увеличивает, на запад уменьшает `X`, на восток — увеличивает. Список `CLOCKWISE` задаёт порядок направлений по часовой стрелке, что удобно для расчётов поворотов (Листинг 7)

```
DIR2ROT: dict[Direction, int] = {
    Direction.N: 0,
    Direction.S: 180,
    Direction.W: -90,
    Direction.E: 90,
}

DIR2OFFSET: dict[Direction, QPoint] = {
    Direction.N: QPoint(0, -1),
    Direction.S: QPoint(0, 1),
    Direction.W: QPoint(-1, 0),
    Direction.E: QPoint(1, 0),
}

CLOCKWISE = [Direction.N, Direction.E, Direction.S, Direction.W]
```

Python

Листинг 7. Направления и смещения

2.3.2. Утилиты

`getMediaPath` возвращает абсолютный путь к медиафайлу (изображению) по его имени. `getPrettyTimestamp` преобразует временную метку в человекочитаемый формат для логирования событий (Листинг 8)

```
def getMediaPath(media_name: str) -> str:
    return str(MEDIA_PATH / media_name)

def getPrettyTimestamp(timestamp: float) -> str:
    return datetime.fromtimestamp(timestamp).strftime(
        "%Y-%m-%d %H:%M:%S"
    )
```

Python

Листинг 8. Вспомогательные функции

Функции преобразования координат: `scenePosToCell` переводит координаты сцены в клетку с округлением вниз (используется для определения клетки под курсором мыши), `snapScenePosToCell` — с математическим округлением, `cellToScenePos` выполняет обратное преобразование из клетки в пиксельные координаты сцены (Листинг 9)


```

def scenePosToCell(pos: QPoint | QPointF) -> QPoint:
    return QPoint(
        math.floor(pos.x() / CELL_SIZE),
        math.floor(pos.y() / CELL_SIZE),
    )

def snapScenePosToCell(pos: QPoint | QPointF) -> QPoint:
    return QPoint(
        round(pos.x() / CELL_SIZE),
        round(pos.y() / CELL_SIZE),
    )

def cellToScenePos(cell: QPoint) -> QPoint:
    return QPoint(
        cell.x() * CELL_SIZE,
        cell.y() * CELL_SIZE,
    )

```

Python

Листинг 9. Функции для преобразование координат

`getDirectionFromOffset` возвращает направление по вектору смещения. `rotateDirection` вычисляет новое направление после выполнения манёвра (поворот налево, направо, разворот или движение прямо), используя порядок `CLOCKWISE` и словарь дельт (Листинг 10)

```

def getDirectionFromOffset(offset: QPoint) -> Direction:
    for direction, dir_offset in DIR2OFFSET.items():
        if offset == dir_offset:
            return direction
    return Direction.N

def rotateDirection(
    direction: Direction, turn: TurnType
) -> Direction:
    delta = {
        TurnType.FORWARD: 0,
        TurnType.RIGHT: 1,
        TurnType.UTURN: 2,
        TurnType.LEFT: -1,
    }
    return CLOCKWISE[
        (CLOCKWISE.index(direction) + delta[turn]) % 4
    ]

```

Python

Листинг 10. Функции для работы с направлением

2.4. Реализация сцены — класс World

`World` наследуется от `QGraphicsScene` и выступает контейнером, хранящим все сущности сцены, а также предоставляет методы для их поиска, навигации и генерации маршрутов. При инициализации задаются цвет фона и сетки, устанавливается прямоугольник сцены с центром в нулевых координатах, а также создаётся и запускается таймер симуляции, который с интервалом `tickMs` управляет поведением всех сущностей.

2.4.1. Поиск и навигация по сцене

`World` предоставляет три основных метода для получения сущностей (Листинг 11). `entities` возвращает все объекты указанного типа (или типов). `get` возвращает сущности в конкретной клетке. `getNeighbors` — сущности, находящиеся в соседних клетках согласно предопределённым направлениям (`DIR2OFFSET`)

```
def entities[T: BaseEntity](
    self, types: type[T] | tuple[type[T]]
) -> list[T]:
    return [e for e in self.items() if isinstance(e, types)]

def get[T: BaseEntity](
    self, at: QPoint, types: type[T] | tuple[type[T]] = BaseEntity
) -> list[T]:
    return [e for e in self.entities(types) if e.cell == at]

def getNeighbors[T: BaseEntity](
    self, at: QPoint, types: type[T] | tuple[type[T]] = BaseEntity
) -> list[T]:
    neighbors = []
    for _, offset in DIR2OFFSET.items():
        neighbors.extend(self.get(at + offset, types))
    return neighbors
```

Python

Листинг 11. Методы получения сущностей

2.4.2. Обработка нажатия по сцене

При клике на сцене `mousePressEvent` (Листинг 12) преобразует координаты события в клетку с помощью `scenePosToCell` и испускает сигнал `onWorldCellClicked`, который обрабатывается в `MainWindow`.

```
def mousePressEvent(  
    self, event: QGraphicsSceneMouseEvent  
) -> None:  
    cell = scenePosToCell(event.scenePos())  
    self.onWorldCellClicked.emit(cell)  
    return super().mousePressEvent(event)
```

Python

Листинг 12. Метод `mousePressEvent`

2.4.3. Отрисовка сетки

`drawBackground` (Листинг 13) рисует сетку: сначала заливает фон, затем линиями размечает клетки с шагом `CELL_SIZE` в пределах видимой области.

```
def drawBackground(self, painter: QPainter, rect: QRectF) -> None:  
    painter.fillRect(rect, self.backgroundColor)  
    painter.setPen(QPen(self.gridColor, 0.4))  
  
    left, right = int(rect.left()), int(rect.right())  
    top, bottom = int(rect.top()), int(rect.bottom())  
    first_left = left - (left % CELL_SIZE)  
    first_top = top - (top % CELL_SIZE)  
  
    for x in range(first_left, right, CELL_SIZE):  
        painter.drawLine(x, top, x, bottom)  
    for y in range(first_top, bottom, CELL_SIZE):  
        painter.drawLine(left, y, right, y)
```

Python

Листинг 13. Метод `drawBackground`

2.4.4. Обновление сущностей по таймеру

Каждый тик таймера вызывает `onTick` (Листинг 14), который проходит по всем сущностям сцены и для каждой вызывает метод `tick`, передавая ему прошедшее время в секундах.

```
def onTick(self) -> None:  
    for entity in self.entities():  
        entity.tick(self.simTimer.interval() / 1000)
```

Python

Листинг 14. Метод `onTick`

2.4.5. Генерация случайного маршрута

Метод `generateRandomPath` строит маршрут от заданной стартовой дороги длиной не более `maxLength` клеток. Алгоритм последовательно движется вперёд, обрабатывая два типа дорог: прямые участки и перекрёстки.

Маршрут начинается со стартовой дороги. Переменная `current` хранит текущую позицию на траектории (Листинг 15)

```
def generateRandomPath(  
    self, start: StraightRoad, maxLength: int = 60  
) -> list[Road]:  
    path: list[Road] = [start]  
    current = start
```

Python

Листинг 15. Инициализация генерации маршрута

На каждой итерации алгоритм запрашивает у сцены дорогу впереди (`getCellOffset(1)` возвращает смещение на одну клетку вперёд относительно направления текущей дороги). Если впереди нет дороги, маршрут прерывается (Листинг 16)

```
for _ in range(maxLength):  
    ahead = self.get(current.getCellOffset(1), Road)  
    if not ahead:  
        break  
    ahead = ahead[0]
```

Python

Листинг 16. Основной цикл генерации маршрута

Если впереди прямая дорога, проверяется совпадение направлений: поворот на прямой дороге невозможен. При успешной проверке дорога добавляется в маршрут, и движение продолжается. (Листинг 17)

```
if isinstance(ahead, StraightRoad):  
    if current.direction != ahead.direction:  
        break  
    path.append(ahead)  
    current = ahead  
    continue
```

Python

Листинг 17. Обработка прямой дороги

При достижении перекрёстка алгоритм получает все возможные варианты проезда через функцию `expandCrossroadPaths`, которая возвращает последовательности смещений для каждого типа поворота (прямо, налево, направо, разворот). Варианты

перемешиваются случайным образом. Для каждого варианта проверяется, существует ли на выходе прямая дорога и можно ли её проехать (`canBePassed`) (Листинг 18)

```
if isinstance(ahead, CrossRoad):
    expandedPaths = list(
        expandCrossroadPaths(current.direction).items()
    )
    random.shuffle(expandedPaths)

    for turnType, pathSteps in expandedPaths:
        dst = self.get(current.cell + pathSteps[-1], StraightRoad)
        if not dst or not dst[0].canBePassed():
            continue

        for stepOffset in pathSteps:
            road = self.get(current.cell + stepOffset, Road)
            if not road:
                break # if gap in path then abort turn
            path.append(road[0])

        current = dst[0]
        break
```

Python

Листинг 18. Обработка перекрестка

Если выходная дорога доступна, алгоритм последовательно добавляет в маршрут все клетки перекрёстка согласно предвычисленным смещениям. Текущей позицией становится выездная дорога, после чего цикл завершается (перекрёсток пройден). Метод возвращает построенный маршрут.

2.5. Базовый класс сущности — BaseEntity

BaseEntity наследуется от QGraphicsPixmapItem и служит фундаментом для всех объектов на сцене (дорог, машин, светофоров и т.д.). Он инкапсулирует клеточную позицию, обработку перемещения и валидацию размещения.

2.5.1. Инициализация

Конструктор (Листинг 19) принимает два параметра: `zIndex` (порядок отрисовки) и `movable` (возможность перемещать объект мышью). По умолчанию все сущности являются выделяемыми, а если `movable` равен `True`, им также добавляется флаг перемещения. Начальная клетка определяется преобразованием текущей позиции в координаты сетки с помощью `scenePosToCell`.

```
class BaseEntity(QGraphicsPixmapItem):
    def __init__(
        self, zIndex: int = 0, movable: bool = True
    ) -> None:
        super().__init__()

        flags = QGraphicsItem.GraphicsItemFlag.ItemIsSelectable
        if movable:
            flags |= QGraphicsItem.GraphicsItemFlag.ItemIsMovable
        self.setFlags(flags)

        self.setZValue(zIndex)
        self.cell = scenePosToCell(self.pos())
```

Python

Листинг 19. Конструктор BaseEntity

2.5.2. Поведение по таймеру

Метод `tick` (Листинг 20) вызывается каждым таймером сцены (передается время в секундах, прошедшее с предыдущего вызова). В базовом классе он ничего не делает и предназначен для переопределения в дочерних классах, где реализуется специфическое поведение — движение машин, переключение светофоров и т.д.

```
def tick(self, dt: float) -> None:
    pass
```

Python

Листинг 20. Метод tick

2.5.3. Обработка перемещения

При отпускании кнопки мыши после перетаскивания сущности на `mouseReleaseEvent` (Листинг 21) координаты курсора преобразуются в клетку с помощью `snapScenePosToCell`. Если новая клетка совпадает со старой или размещение в ней неважно, сущность возвращается на прежнюю позицию. В противном случае выполняется перемещение.

```
def mouseReleaseEvent(  
    self, event: QGraphicsSceneMouseEvent  
) -> None:  
    super().mouseReleaseEvent(event)  
    cell = snapScenePosToCell(self.pos())  
  
    if cell == self.cell or not self.validatePlacement(cell,  
self.world):  
        self.setCell(self.cell)  
        return  
  
    self.setCell(cell)
```

Python

Листинг 21. Метод-событие `mouseReleaseEvent`

2.5.4. Установка позиции

Метод `setCell` (Листинг 22) обновляет внутреннее представление клетки (`self.cell`) и устанавливает графическую позицию сущности на сцене, преобразуя координаты клетки в пиксельные через `cellToScenePos`.

```
def setCell(self, cell: QPoint) -> None:  
    self.cell = cell  
    self.setPos(cellToScenePos(cell))
```

Python

Листинг 22. метод `setCell`

2.5.5. Валидация размещения

Метод `validatePlacement` (Листинг 23) проверяет, можно ли разместить сущность в заданной клетке. По умолчанию запрещается размещение двух сущностей одного типа в одной клетке. Дочерние классы могут переопределять этот метод для более сложных правил.

```
def validatePlacement(self, cell: QPoint, world: "World") -> bool:  
    existingItems = world.get(cell, types=type(self))  
    return len(existingItems) == 0
```

Python

Листинг 23. Метод `validatePlacement`

2.6. Сериализация и загрузка сцены – модуль `persistence`

Модуль `persistence` отвечает за сохранение состояния сцены в JSON-файл и восстановление из него. Это позволяет пользователю сохранять созданные сцены дорожной сети и загружать их при следующем запуске.

2.6.1. Сериализаторы сущностей

`Serializer` (Листинг 24) — это дженерик-датакласс, который связывает пару функций для конкретного типа сущности: `serialize` преобразует объект в словарь, а `deserialize` восстанавливает объект из словаря.

```
@dataclass
class Serializer[T: BaseEntity]:
    serialize: Callable[[T], dict]
    deserialize: Callable[[dict[str, Any]], T]
```

Python

Листинг 24. Dataclass сериализатора сущностей

2.6.2. Регистр сериализаторов

Словарь `SERIALIZERS` (Листинг 25) сопоставляет каждый класс сущности с его сериализатором. Для дорог сохраняется направление, для знаков — тип, для пешеходных переходов — ориентация. Перекрёстки, светофоры и пешеходы не имеют дополнительных параметров, поэтому их сериализаторы работают с пустыми словарями.

```
SERIALIZERS: dict[type[BaseEntity], Serializer] = {
    StraightRoad: Serializer(
        lambda e: {"direction": e.direction.name},
        lambda d: StraightRoad(
            Direction[d.get("direction", "N")]
        ),
    ),
    ...
}
```

Python

Листинг 25. Регистр сериализаторов

`CLASS_MAP` (Листинг 26) используется для обратного преобразования: по строковому имени класса (например, `"StraightRoad"`) получается соответствующий класс Python.

```
CLASS_MAP: dict[str, type[BaseEntity]] = {
    cls.__name__: cls
    for cls in SERIALIZERS
}
```

Python

Листинг 26. Карта классов

2.6.3. Сериализация сущности

Функция `serializeEntity` (Листинг 27) принимает сущность и возвращает словарь с четырьмя полями:

- Тип сущности
- X клетки
- Y клетки
- Данные необходимые для инициализации - результат работы `serialize`

Если для типа нет сериализатора, выбрасывается исключение.

```
def serializeEntity(entity: BaseEntity) -> dict:
    serializer = SERIALIZERS.get(type(entity))
    if not serializer:
        raise ValueError(f"No serializer for
{type(entity).__name__}")

    return {
        "type": type(entity).__name__,
        "x": entity.cell.x(),
        "y": entity.cell.y(),
        "data": serializer.serialize(entity),
    }
```

Python

Листинг 27. Функция `serializeEntity`

2.6.4. Десериализация сущности

`deserializeEntity` (Листинг 28) выполняет обратное преобразование: по полю `"type"` определяется класс, извлекается соответствующий сериализатор, создаётся объект без позиции (через `deserialize`), а затем позиция устанавливается отдельным вызовом `setCell`.

```
def deserializeEntity(entityData: dict[str, Any]) -> BaseEntity:
    cls = CLASS_MAP[entityData["type"]]
    serializer = SERIALIZERS.get(cls)

    if not serializer:
        raise ValueError(f"No deserializer for {cls.__name__}")

    obj = serializer.deserialize(entityData.get("data", {}))
    obj.setCell(QPoint(entityData["x"], entityData["y"]))

    return obj
```

Python

Листинг 28. Функция `deserializeEntity`

2.6.5. Сохранение сцены

`saveWorld` (Листинг 29) собирает все сущности со сцены, сериализует каждую через `serializeEntity` и записывает JSON-объект с корневым полем `"entities"` в указанный файл.

```
def saveWorld(world: World, path: str) -> None:
    entities = [
        serializeEntity(entity)
        for entity in world.entities()
    ]

    with open(path, "w", encoding="utf-8") as f:
        json.dump({"entities": entities}, f, indent=2)
```

Python

Листинг 29. Функция `saveWorld`

2.6.6. Загрузка сцены

`loadWorld` (Листинг 30) открывает JSON-файл, очищает текущую сцену и добавляет каждую десериализованную сущность на сцену.

```
def loadWorld(world: World, path: str) -> None:
    with open(path, "r", encoding="utf-8") as f:
        worldData = json.load(f)

    world.clear()

    for entity in worldData["entities"]:
        world.addItem(deserializeEntity(entity))
```

Python

Листинг 30. Функция `loadWorld`